IronFit Movement Mirror — Concept, Brief & Build Rationale

A companion document to the Trainer Guide: what we set out to build, why, the thinking and domain research behind it, and how the prototype came together.

The original brief

A summary of what was asked for. The complete original prompt is reproduced verbatim in the Appendix at the end of this document.

Build a **mobile-first web app that helps IronFit clients rehearse Functional Range Systems (FRS) movements between sessions.** The trainer (Lee, FRC/FRA certified) records reference videos of each movement; the app extracts joint-angle data from those videos and then gives clients **live, joint-by-joint feedback** while they perform the same movement in front of their phone camera.

The guiding constraints in the brief were explicit and unusually principled:

- **"It's a mirror with a memory, not a coach."** The trainer defines what correct looks like; the app helps the client rehearse it accurately. The app must not invent movements, prescribe loads, or progress anyone — the trainer does that.
- **Privacy first.** No client video or pose data should ever leave the device.
- **Honesty over hype.** Be explicit about what the technology can and can't do, flag low-confidence readings rather than fake precision, and *"tell me what will break before I have to find out."*
- **Real performance targets** on real phones (hold ~24fps live; warn below 15).
- **A coaching tool, not a medical device** — it does not diagnose injury or replace in-person assessment.

This document explains the reasoning that turned that brief into the working prototype.

The problem it solves

FRS training (CARs, PAILs/RAILs, controlled articular work) lives or dies on **precision between sessions.** A client sees their trainer occasionally, then goes home and rehearses — often drifting from the exact range and control the trainer prescribed, with no feedback until the next session weeks later. Cues get forgotten; compensations creep in; progress stalls and nobody sees why.

Generic fitness apps don't help here because they optimize for the wrong thing: streaks, points, calories, and rep counts. FRS isn't about *how many* — it's about *how well*: did the joint actually reach end range, was it controlled, did the body cheat the movement with a compensation. The brief demanded a tool built around **range and control**, measured against the trainer's own standard.

Domain grounding & research

The design is grounded in how FRS is actually coached, and in an honest read of what consumer phone-based pose estimation can and cannot measure.

FRS coaching concepts the app models directly

- **Movement types** — CAR (controlled articular rotation), PAILs/RAILs, flows.
- **Range of motion (ROM)** measured per joint against a clean reference rep.

- **Compensation patterns** — the specific ways a body cheats a movement (e.g. lumbar substitution, scapular hiking), each paired with a coaching cue.
- **The trainer's reference is the standard.** There is no "ideal" baked in; the app compares the client to the trainer's own recorded range, not a textbook.

Honest limits of 2D pose estimation (analysis done up front)

A single phone camera with a 2D pose model (MediaPipe) sees joint *positions*, not twist around a limb's long axis. Rather than paper over this, we mapped exactly where it would be weak and built the honesty in:

- **Rotation-dominant movements** (shoulder/hip internal-external rotation, forearm pronation/supination) are flagged **approximate** and never shown as a confident reading; the app nudges the user to also film from the side.
- **Ankle / foot-dependent ranges** are capped at "reduced" confidence — feet are the noisiest landmarks.
- **Thoracic vs lumbar spine** can't be separated by a single-camera proxy, so a spinal flag means "the trunk moved," not a segment-specific claim.
- Every joint carries a **confidence badge**; color is never the only signal.

Thresholds: defensible defaults, not invented standards

FRS does not publish numeric app tolerances. So every threshold (what counts as "green" vs "short," what drift counts as a compensation, rep-detection sensitivity) is a **documented, tunable default for the trainer to review** — centralized in one file with its reasoning and a confidence rating, explicitly flagging the ones most in need of real-world tuning. We did not dress guesses up as science.

Key design decisions & the reasoning behind each

1. On-device processing, privacy by architecture. The same pose model and angle math run in the browser on both sides. Client video is analyzed on their phone and discarded; only the *scores* (ROM %, compensation flags, a confidence rating) are stored. This isn't a setting that can be misconfigured — there is no server-side video pipeline to leak.

2. "Mirror with a memory," not an AI coach. The app deliberately does not generate programs or progress clients. It rehearses what the trainer prescribed. This keeps the trainer central, keeps the scope honest, and avoids the failure mode of an app confidently coaching something it can't actually assess.

3. Baseline + running-peak comparison, no tempo-syncing. Time-syncing a client to the trainer's exact tempo is brittle. Instead each joint locks a baseline on the first good frame, tracks the running peak, and compares *range achieved* to the reference range. A client moving at half speed still gets honest end-range feedback — and the summary says so out loud.

4. Non-vanity coaching metrics. The dashboards answer coaching questions, not engagement ones. The system surfaces four signals worth a trainer's attention: a **recurring compensation**, **range that's stalled or declining**, a client who's **gone quiet**, and a **camera setup that's hurting feedback quality**. No streaks, points, or totals-for-their-own- sake.

5. Intake-link client assignment. The earliest version could only attach a client to a trainer by hand (a SQL update) — a known gap. The prototype replaces it: each trainer has a unique intake link; a client fills a short intake form and is **auto-assigned to that trainer**, with their goals and injury history captured up front. This also fixed a real bug where every new client defaulted to a single trainer.

6. Library + "Your focus." Clients see the full movement library, but trainers can pin a few movements as that client's focus (with a note), highlighted at the top. It balances autonomy with direction.

7. Practice-wide overview shared by all trainers. For a small multi-trainer practice, all trainers share one overview of every roster, who needs attention, and the ability to reassign clients — chosen over a rigid owner-only admin tier to match how a small team actually operates.

8. Scanner-proof authentication. Auth evolved deliberately: magic link !' 6-digit email code !' finally **email + password with no email in the login path at all**, so corporate email scanners, rate limits, and delivery failures can't block sign-in.

How it was built — iteration timeline

1. **Core libraries first.** Pose detection, reference extraction, and live comparison were built as pure, unit-tested modules before any UI.
2. **The app shell.** Trainer and client flows, Supabase wiring, row-level security, and the live performance overlays.
3. **Honest documentation.** A developer + trainer guide, the recording guide, and a frank "what I expect to break" section.
4. **Production hardening.** Self-contained build, connected to its own database, and the authentication rework for reliable sign-in.
5. **The coaching layer.** Trainer and client dashboards with the non-vanity metrics, plus in-app realtime messaging.
6. **The operating model.** Intake-link onboarding, per-client programs, the practice-wide overview, and the trainer guide — turning a single-trainer demo into a real multi-trainer practice tool.

Technology & architecture

- **Frontend:** React + Vite (TypeScript), mobile-first, deployed on Vercel.
- **Pose & analysis:** MediaPipe Tasks Vision, running entirely in the browser. Reference extraction (trainer) and live comparison (client) share identical angle math, so they're directly comparable — and there is no server-side ML.
- **Backend:** Supabase (Postgres) with row-level security enforcing that clients see only their own data and trainers see their practice. Private storage buckets for reference videos with signed playback URLs.
- **Quality:** a unit-test suite over the geometry, extraction, and comparison logic, plus end-to-end smoke tests.

A deliberate scalability note was built in: in-browser extraction is right for now, but the extractor core is pure and decoupled, so it can be lifted into a serverless worker the moment the trainer is processing many or long clips.

Status & roadmap

The prototype is a **working multi-trainer practice tool**: trainers build movements, recruit and onboard clients via intake links, program each client's focus, coach from the dashboards, message clients, and share a practice-wide overview. Clients rehearse with live feedback and watch their range improve.

Intentionally deferred to last, per plan:

- **Billing** — kept out until the rest of the app is settled; clients currently see a neutral "handled by your trainer" placeholder.

- **Known follow-ups** documented honestly: audio cues (text-to-speech of the trainer's cues), auto-generated captions, per-rep breakdowns, left/right symmetry scoring, and moving reference extraction to a worker as volume grows.

The whole thing is built to grow: principled scope, honest limits, and a clean separation between what the technology measures and what the trainer decides.

Appendix — The Original Prompt

The complete original build brief, reproduced verbatim as written.

Build: IronFit Movement Mirror (v1, lean)

What this is

A mobile-first web app for IronFit clients. The trainer (Lee, FRC/FRA certified) records reference videos of Functional Range Systems movements: CARs (Controlled Articular Rotations), PAILs/RAILs (Progressive/Regressive Angular Isometric Loading), and Kinstretch flows. Clients open the app, pick a movement, watch Lee's reference, then perform the movement in front of their phone camera. The app analyzes the client's live performance against the reference and shows them, in real time, which joints are tracking the movement correctly and which are compensating.

This is a movement coaching mirror, not a tracker. The goal is teaching the client to feel the right movement by giving them honest, immediate visual feedback while they do it.

Core principle (non-negotiable)

AI amplifies the trainer, never replaces them. Lee defines what correct looks like. The app helps the client rehearse it accurately between sessions. The app does not invent movements, does not prescribe loads, does not progress the client. Lee does that. The app is a mirror with a memory.

Honest constraint up front

2D pose detection (MediaPipe Pose) captures joint positions reliably but is weak on rotation around a limb's long axis. Shoulder internal/external rotation, hip internal/external rotation, and forearm pronation/supination are the most affected. The app must surface this honestly: when a movement depends on rotation that the camera can't see well, the app says "rotation axis assessment is approximate from this angle" rather than faking a confident reading.

Scope (lean v1)

- Trainer-side: upload reference video, tag movement metadata, app extracts reference joint data automatically
- Client-side: browse movement library, watch reference video, perform movement in front of camera with live joint-by-joint feedback
- 5-15 movements at launch (Lee picks)
- Single trainer (Lee), unlimited clients
- No payments, no scheduling, no progress tracking beyond "completed today" in v1
- No social features
- iOS Safari and Android Chrome
- Web app, not native. Built for installable PWA later.

Stack

- Vite + React + TypeScript

- Tailwind CSS
- MediaPipe Pose Landmarker via @mediapipe/tasks-vision, VIDEO mode, full model variant, GPU delegate
- Canvas API for live joint overlay
- Supabase for: movement metadata, reference joint data, video storage, trainer/client auth
- Supabase Auth (email magic link, no passwords)
- Vercel deploy
- No native app shell in v1

Build complete. No partial ship.

Trainer upload flow works end to end. Reference extraction works. All 5-15 launch movements processed and queued. Client live feedback runs at 24fps minimum on a 3-year-old phone. Compensation detection works for every movement Lee tags. Honest confidence labeling on every joint. Full test suite. Accessibility WCAG 2.1 AA. README that Lee can follow without asking questions.

Use npm. After scaffolding, run the test suite and confirm all tests pass. Start the dev server with `--host 0.0.0.0` so I can reach it from my phone. Give me the local network URL. Stop and wait for me to test before deploying.

User flows

Trainer flow (Lee)

1. Sign in with magic link.

2. Trainer dashboard: list of movements, "Add Movement" button.

3. Add Movement form: - Movement name (e.g. "Shoulder CAR, right") - Movement type: CAR, PAILs, RAILs, Kinstretch flow, other - Primary joint (dropdown: shoulder, elbow, wrist, hip, knee, ankle, cervical, thoracic, lumbar) - Side: left, right, bilateral, midline - Recommended camera angle: front, side, 45°, overhead, behind - Target joints (multi-select): which joints should be moving - Stillness joints (multi-select): which joints should NOT move (compensation watch) - ROM expectation: estimated peak angle to achieve (degrees, optional) - Tempo expectation: slow/moderate/fast or seconds per rep - Cues: text field, the verbal cues Lee gives ("drive the elbow forward," "keep the ribcage down") - Compensation patterns to flag: free text + structured tags ("lumbar extension," "scapular elevation," "thoracic flexion")

4. Upload reference video (mp4, up to 60 seconds, up to 100MB).

5. App processes video: - Runs pose detection frame by frame - Extracts joint angle trajectories over time for all target and stillness joints - Computes peak ROM achieved on target joints - Computes movement variance on stillness joints (should be low) - Computes tempo (cycle time per rep) - Stores reference dataset

6. Lee reviews the extracted reference: - Sees plotted joint angle curves - Sees ROM peaks - Approves or re-records

7. Movement is now live for clients.

Client flow

1. Sign in with magic link (Lee invites them).

2. Movement library: grid of movements with thumbnails. Filter by joint.

3. Tap a movement.

4. Movement detail screen: - Lee's reference video (autoplay, muted, loop) - Movement name, primary joint, Lee's cues - "Try it" button

5. Tap Try it: - Camera permission prompt - "Position your phone: [recommended camera angle]. Distance: full body in frame." - 3-second countdown

6. Live performance screen: - Live camera view, full screen - Skeleton overlay - Target joints highlighted in moving color (green when moving correctly, yellow when partial, red when not moving

or moving wrong direction) - Stillness joints highlighted in static color (blue when still, red when moving) - Compensation pattern alerts pop up as banner ("Lumbar extending — drop the ribs") - Live ROM meter for the primary joint: arc showing achieved range vs reference peak - "Done" button

7. Summary screen: - Side-by-side: Lee's reference joint curve vs client's joint curve - ROM achieved: X° vs reference Y° (Z% of reference) - Compensation patterns detected: list with timestamps - Stillness joints: which ones moved when they shouldn't have - Confidence note: any joints where the assessment was rotation-limited - "Try again" or "Done"

8. Local history: last 10 attempts per movement saved to LocalStorage. Just timestamps and ROM percentages, no video.

Reference extraction (the heart of trainer side)

When Lee uploads a video, the server-side or client-side processor (decide: do this in browser using same MediaPipe model so we don't need a server-side ML pipeline in v1) runs:

1. Decode video to frames at 30fps (or native framerate if lower).

2. Run pose detection on each frame. Store all 33 landmarks per frame with visibility scores.

3. Compute joint angles for every relevant joint at every frame: - Shoulder flexion/extension: angle between torso vector and humerus vector in sagittal plane - Shoulder abduction/adduction: same but coronal plane - Shoulder internal/external rotation: approximate from elbow position relative to shoulder when arm abducted. Flag as low-confidence in absence of side angle. - Elbow flexion: angle at elbow between humerus and forearm vectors - Hip flexion/extension: torso to femur sagittal angle - Hip abduction/adduction: torso to femur coronal angle - Hip internal/external rotation: approximate from knee position. Flag as low-confidence. - Knee flexion: femur to tibia angle - Ankle dorsiflexion: tibia to foot angle (foot landmarks have low confidence; flag) - Cervical flexion/extension/rotation: head landmark relative to shoulder line - Spinal flexion/extension proxy: shoulder-mid to hip-mid angle relative to vertical - Spinal rotation proxy: shoulder line vs hip line angular difference

4. For each target joint (per movement metadata), compute: - Trajectory over time (array of angle values) - Peak angle achieved - ROM (max minus min) - Smoothness (variance from a smoothed curve)

5. For each stillness joint, compute: - Angular variance over time (should be small) - Maximum deviation from starting position

6. Compute movement cycle: detect repetitions if Lee marked the video as a multi-rep movement. Use peak detection on the primary joint angle trajectory.

7. Store the full reference dataset as JSON in Supabase, linked to the movement record.

The reference JSON shape:

```
type ReferenceDataset = {
  movement_id: string;
  fps: number;
  duration_sec: number;
  rep_count: number;
  target_joints: {
    [joint_name: string]: {
      trajectory: number[];       // angle per frame
      peak: number;
      rom: number;
      smoothness_score: number;
      confidence: 'high' | 'reduced' | 'low';
    };
  };
  stillness_joints: {
    [joint_name: string]: {
      baseline_angle: number;
      max_deviation: number;
      variance: number;
      confidence: 'high' | 'reduced' | 'low';
    };
  };
  cues: string[];
  compensation_patterns: string[];
};
```

Live comparison (the heart of client side)

While the client performs the movement, run pose detection at 24fps minimum. For each frame:

1. Compute the same joint angles as the reference for all target and stillness joints.
2. For each target joint, compare the live angle to the reference trajectory: - If the client's joint is within 15% of the expected angle at this cycle phase: GREEN - 15-30% off: YELLOW - Over 30% off, or moving in the wrong direction: RED
3. For each stillness joint, compare deviation from the client's starting position: - Under 5° deviation: BLUE (still) - 5-15°: YELLOW (drifting) - Over 15°: RED (compensating, fire the compensation pattern alert)
4. Render the joint highlights on the skeleton overlay with these colors.
5. ROM meter for the primary target joint: render an arc showing current achieved peak vs reference peak. Updates in real time.
6. Compensation alerts: when a stillness joint exceeds threshold, surface the matching compensation pattern from movement metadata as a banner with the cue text Lee provided.

Temporal alignment is hard. v1 approach: don't try to sync to Lee's exact tempo. Instead, track the client's own peak and compare ROM achieved, not phase-matched angles. So a slow client doing a shoulder CAR at half Lee's speed still gets accurate feedback on whether they're hitting end range.

Confidence handling

Every joint angle has a confidence based on landmark visibility scores:

- High: all required landmarks above 0.7 visibility
- Reduced: some landmarks 0.5-0.7
- Low: any required landmark below 0.5

Reduced confidence shows the joint in a paler color. Low confidence skips evaluation and shows the joint in gray with a small "?" indicator.

Rotation axis joints (shoulder IR/ER, hip IR/ER, forearm pronation/supination) are flagged as approximate when the camera angle is front-on. The summary screen tells the user explicitly: "Shoulder internal rotation was assessed approximately. For best accuracy, also try this movement from a side angle."

Camera angle validation

When client taps "Try it":

1. Show recommended angle from movement metadata.
2. Show a live preview.
3. App estimates current camera angle from shoulder-width to body-length ratio (same approach as the calibration math you've seen elsewhere).
4. Compare to recommended.
5. If significantly off, show "Adjust position: turn 90°" or similar.
6. Allow override but flag the session with reduced confidence.

Trainer dashboard details

- Movement list with thumbnail, name, primary joint, status (Draft, Live, Archived)
- Click a movement to edit metadata or re-upload reference video
- "Reference quality" indicator per movement: high if all target joints extracted at high confidence, reduced if any are reduced, low if any are low
- Reference video player with overlay of extracted joint trajectories so Lee can visually verify the extraction worked

- Bulk: cannot bulk-edit in v1

Client dashboard details

- Greeting + last completed movement
- Movement library grid with thumbnails
- Filter by joint (chips: Shoulder, Elbow, Hip, Knee, Ankle, Spine, Cervical)
- Filter by type (chips: CAR, PAILs, RAILs, Flow)
- Recently attempted at top

Data model (Supabase)

```
-- users: handled by Supabase auth, with a role column on profiles
-- profiles: user_id, role ('trainer' | 'client'), display_name, created_at

-- movements
create table movements (
  id uuid primary key default gen_random_uuid(),
  trainer_id uuid references profiles(user_id),
  name text not null,
  movement_type text not null check (movement_type in ('car', 'pails', 'rails',
'flow', 'other')),
  primary_joint text not null,
  side text not null check (side in ('left', 'right', 'bilateral', 'midline')),
  recommended_camera_angle text not null,
  target_joints jsonb not null,
  stillness_joints jsonb not null,
  rom_expectation_deg numeric,
  tempo_expectation text,
  cues text[] default '{}',
  compensation_patterns jsonb default '[]',
  reference_video_path text,
  reference_dataset jsonb,
  reference_quality text check (reference_quality in ('high', 'reduced', 'low')),
  status text not null default 'draft' check (status in ('draft', 'live',
'archived')),
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

-- client_attempts (lean: just timestamp + ROM achieved, no video stored)
create table client_attempts (
  id uuid primary key default gen_random_uuid(),
  client_id uuid references profiles(user_id),
  movement_id uuid references movements(id),
  rom_achieved_pct numeric,
  compensation_flags text[] default '{}',
  confidence text,
  attempted_at timestamptz default now()
);
```

Row-level security:

- Trainers can read/write their own movements
- Clients can read all live movements from their trainer
- Clients can read/write their own attempts
- Trainers can read attempts of their clients

Storage

- Reference videos: Supabase Storage, private bucket, signed URLs for client playback
- No client performance video stored anywhere. Pose data is processed in-browser and discarded after the session.
- Thumbnails generated from first frame of reference video on upload

Privacy

- No client video leaves the device
- Client attempt records store only ROM percentages and compensation flags, no biometric raw data
- Trainer can see client attempt history; client can delete their own history
- Footer: "IronFit Movement Mirror is a coaching tool. It does not diagnose injury or replace in-person assessment by a qualified practitioner."

Edge cases

- Client video has no person visible: error, prompt to reposition
- Camera angle dramatically wrong: warning, allow override with confidence flag
- Frame rate drops below 15fps: warning ("Performance is limited on this device. Feedback may be delayed.")
- Lee's reference video has low pose detection confidence on target joints: reference_quality marked low, banner on trainer dashboard prompts re-shoot
- Client interrupted mid-movement (phone call, notification): pause and resume cleanly
- Network drops during reference upload: resumable upload via Supabase tus
- Trainer edits a movement after clients have attempted it: existing attempts remain, new attempts use new reference

Accessibility (WCAG 2.1 AA)

- All controls keyboard-accessible
- Color is never the only indicator. Joint colors are paired with text labels in the side panel ("Right hip: moving correctly")
- Audio cues option (toggle in settings): app speaks Lee's verbal cues at appropriate moments
- Reduced motion respected on transitions
- Closed captions on Lee's reference videos (Lee provides or auto-generates)
- Color contrast meets AA

Testing

- Vitest unit tests on geometry/joint angle math
- Vitest unit tests on the comparison logic with fixture reference datasets
- Synthetic fixtures: a perfect-match performance, a 25% under-ROM performance, a compensating performance
- Playwright smoke test: trainer signs in, uploads a fixture video, sees extracted reference; client signs in, picks the movement, sees live screen
- Manual test checklist in README for iOS Safari and Android Chrome covering: camera permission, frame rate, joint highlighting, compensation alerts, ROM meter, summary screen

File structure

```
src/
  components/
    common/
      AppShell.tsx
      AuthGate.tsx
      MagicLinkForm.tsx
    trainer/
      TrainerDashboard.tsx
      MovementList.tsx
      MovementEditor.tsx
      ReferenceUploader.tsx
      ReferenceReview.tsx
    client/
```

```
      ClientDashboard.tsx
      MovementLibrary.tsx
      MovementDetail.tsx
      PerformanceScreen.tsx
      SummaryScreen.tsx
      HistoryList.tsx
    overlay/
      SkeletonOverlay.tsx
      JointHighlight.tsx
      RomMeter.tsx
      CompensationBanner.tsx
      ConfidenceIndicator.tsx
  lib/
    pose/
      detector.ts
      landmarks.ts
      angles.ts
      confidence.ts
    reference/
      extractor.ts            // video !' reference dataset
      cycleDetect.ts          // rep detection
      smoothing.ts
    compare/
      liveCompare.ts          // per-frame comparison
      thresholds.ts           // all numeric thresholds with comments
      compensationDetect.ts
    calibration/
      cameraAngle.ts
      bodyHeightNormalize.ts
    supabase/
      client.ts
      auth.ts
      movements.ts
      attempts.ts
      storage.ts
  types/
    movement.ts
    reference.ts
    attempt.ts
    profile.ts
  routes/
    index.tsx
    auth.tsx
    trainer/...
    client/...
  App.tsx
  main.tsx
supabase/
  migrations/
    0001_init.sql
tests/
  pose/
    angles.test.ts
  reference/
    extractor.test.ts
    cycleDetect.test.ts
  compare/
    liveCompare.test.ts
    compensationDetect.test.ts
  fixtures/
    references/
      shoulder-car-clean.json
      shoulder-car-compensating.json
      hip-car-clean.json
  e2e/
    trainer-flow.spec.ts
    client-flow.spec.ts
README.md
```

What I want you to do

1. Scaffold the full project. Vite + React + TS + Tailwind. All files in the structure above.
2. Set up Supabase: create the migration, generate the client, wire auth with magic links, set up row-level security.

3. Implement the pose pipeline: detector wrapper, joint angle math, confidence handling.

4. Implement reference extraction: take an uploaded video, run pose detection, compute trajectories, compute peaks/ROM/variance/smoothness, detect cycles, return a ReferenceDataset.

5. Build the trainer dashboard: movement list, movement editor form, reference upload with progress, reference review screen with trajectory plots.

6. Implement live comparison: per-frame angle computation, comparison against reference, color logic, ROM meter logic, compensation detection.

7. Build the client side: library, movement detail with reference video, performance screen with full overlay (skeleton + colored joints + ROM meter + compensation banner), summary screen with side-by-side trajectory plot.

8. Implement camera angle validation and confidence labeling honestly. Rotation-axis joints labeled approximate when assessed from front view.

9. Build the LocalStorage history for clients (timestamps and ROM percentages only).

10. Write the test suite with synthetic fixture references. All passing.

11. Write the README. Two audiences: developer setup; and Lee, the trainer, with a 5-minute guide to recording a good reference video.

Tell me what breaks before I have to ask. Specifically, tell me:

- Which movements you expect MediaPipe to handle poorly and why
- Which thresholds you're least confident in
- Where in-browser reference extraction will be too slow and need a serverless processing function instead
- Any edge cases I missed

Do not invent thresholds. Where literature or FRS published standards don't exist, propose a defensible default with reasoning so Lee can review and adjust.

Use npm. After scaffolding, run all tests. Start the dev server with `--host 0.0.0.0` and give me the local network URL. Stop and wait for me to test before deploying.